

LocalLLM-DataForge: Un Pipeline Modular para Conjuntos de Datos Sintéticos de Descomposición de Historias de Usuario con Validación tipo LLM-as-a-Judge

Angel Luis Valdés Sánchez¹,
Carlos Alberto Fernández y Fernández^{2*},
Verónica Rodríguez López²

¹División de Estudios de Posgrado, Universidad Tecnológica de la Mixteca, Av. Dr. Modesto Seara Vázquez No. 1, Huajuapán de León, 69004, Oaxaca, México.

^{2*}Instituto de Computación, Universidad Tecnológica de la Mixteca, Av. Dr. Modesto Seara Vázquez No. 1, Huajuapán de León, 69004, Oaxaca, México.

*Corresponding author(s). E-mail(s): caff@gs.utm.mx;
Contributing authors: alvaldes.dev@gmail.com; veromix@gs.utm.mx;

Abstract

La descomposición de historias de usuario en tareas de desarrollo es una práctica habitual en los proyectos ágiles; sin embargo, continúa siendo un proceso altamente dependiente del juicio humano. Los intentos de automatizar esta actividad mediante técnicas de aprendizaje automático suelen enfrentar dos limitaciones recurrentes: la escasez de conjuntos de datos anotados y la dependencia de modelos de lenguaje basados en servicios en la nube. Este trabajo presenta *LocalLLM-DataForge*, un framework modular basado en DataFrames que permite generar conjuntos de datos sintéticos para la descomposición de historias de usuario utilizando modelos de lenguaje ejecutados localmente. El sistema se integra con Ollama para la ejecución de modelos, incorpora mecanismos de verificación automática de calidad inspirados en enfoques de *LLM-as-a-judge* y permite experimentar con diferentes configuraciones de generación y evaluación. La propuesta se evaluó mediante experimentos realizados sobre el conjunto de datos Salony utilizando modelos como Llama 3.1 y Mistral. Se exploraron dos estrategias de evaluación: una basada en la valoración de un único generador a

partir de criterios explícitos de calidad y otra basada en la comparación entre generadores para seleccionar la descomposición más sólida. Los resultados sugieren que las descomposiciones generadas presentan coherencia lógica y una alineación consistente con el criterio de expertos, lo que indica que el uso de LLMs locales puede constituir una alternativa viable para apoyar la generación de datasets en investigación sobre ingeniería de requisitos.

Keywords: Requirements engineering, Natural language processing, Large language models, Synthetic data generation, Task decomposition, Automated evaluation

1 Introducción

Las historias de usuario son uno de los artefactos de requisitos más utilizados en el desarrollo de software ágil. Los equipos suelen apoyarse en ellas para capturar necesidades funcionales desde la perspectiva del usuario final de forma concisa y accesible [1, 2]. En la práctica, su función va más allá de la comunicación: las historias de usuario se emplean de manera habitual para apoyar la planificación, la estimación de esfuerzo y las actividades cotidianas de desarrollo. En particular, la descomposición de las historias en tareas de desarrollo concretas resulta crítica durante la refinación del backlog y la planificación de sprints [3, 4]. Aun así, esta actividad sigue dependiendo en gran medida del trabajo manual y de la experiencia acumulada del equipo.

Cuando la descomposición de tareas se realiza de forma manual, tienden a aparecer una serie de problemas recurrentes. Distintos miembros del equipo pueden interpretar una misma historia de usuario de maneras ligeramente diferentes, lo que da lugar a niveles inconsistentes de granularidad y a supuestos implícitos que nunca llegan a documentarse por completo. Algunos detalles relevantes pueden pasarse por alto (e.g., restricciones funcionales implícitas), mientras que otros aspectos se descomponen en exceso. Con el tiempo, estas inconsistencias contribuyen a la creación de backlogs poco estructurados, lo que dificulta la estimación y la asignación de tareas [5]. El problema suele intensificarse en equipos grandes o distribuidos, donde la ausencia de prácticas compartidas de descomposición debilita la trazabilidad y complica la automatización sistemática de los flujos de trabajo en ingeniería de requisitos [6].

Los avances recientes en Procesamiento del Lenguaje Natural (NLP, por sus siglas en inglés), junto con la rápida evolución de los Modelos de Lenguaje Grande (LLMs, por sus siglas en inglés), han abierto nuevas posibilidades para automatizar tareas que tradicionalmente quedaban en manos de ingenieros de software [7, 8]. Trabajos previos han explorado el uso de LLM para la clasificación de requisitos, la detección de ambigüedades, la generación de casos de prueba y la derivación de tareas de desarrollo a partir de descripciones en lenguaje natural [9, 10]. Sin embargo, la mayoría de las soluciones existentes dependen de modelos propietarios alojados en la nube. Esta dependencia introduce preocupaciones prácticas relacionadas con el costo, la confidencialidad de los datos, la reproducibilidad y la disponibilidad a largo plazo, tanto en contextos de investigación como industriales.

Una limitación compartida por muchos enfoques basados en LLM es la falta de conjuntos de datos adecuados para entrenamiento y evaluación. Los datasets públicos centrados específicamente en la descomposición de historias de usuario siguen siendo escasos, en gran medida debido al elevado costo de la anotación por expertos y a la naturaleza sensible de los requisitos del mundo real [11]. La generación de datos sintéticos mediante LLM se ha propuesto como una solución parcial. No obstante, muchos de los métodos existentes ofrecen poca visibilidad sobre cómo se validan las salidas generadas. Como consecuencia, evaluar la corrección semántica, la completitud o la coherencia lógica resulta complicado [12]. Además, una generación poco controlada incrementa el riesgo de alucinaciones, lo que puede reducir la confianza en los conjuntos de datos resultantes.

Se presenta LocalLLM-DataForge, un framework modular diseñado para generar conjuntos de datos sintéticos orientados a la descomposición de historias de usuario utilizando LLMs ejecutados de forma local. El framework se apoya en DataFrames para estructurar y gestionar los resultados intermedios del proceso de generación. Asimismo, se integra con Ollama para ejecutar modelos de lenguaje de código abierto sin depender de servicios en la nube. Uno de sus componentes centrales es un mecanismo de caché inteligente, que evita cálculos repetidos durante experimentos iterativos y contribuye a reducir los tiempos de ejecución al explorar distintas configuraciones de generación.

El control de calidad se aborda mediante mecanismos de validación automática. LocalLLM-DataForge aplica estrategias de LLM como juez para evaluar las descomposiciones de tareas generadas. Se consideran dos enfoques de evaluación, el primero se basa en un único generador y evalúa sus salidas a partir de criterios de calidad explícitos (i.e., coherencia, completitud, viabilidad, formato y granularidad) [13]. El segundo compara las salidas producidas por múltiples generadores y selecciona la descomposición más adecuada. Esta estrategia comparativa aprovecha principios de consenso que han demostrado mejorar la fiabilidad y reducir errores semánticos en las salidas de los modelos [14, 15].

Este trabajo realiza tres contribuciones principales. En primer lugar, presenta un framework completamente local, extensible y reproducible para la generación de conjuntos de datos sintéticos a partir de historias de usuario, abordando de forma explícita restricciones de costo y privacidad. En segundo lugar, define y operacionaliza estrategias automáticas de validación basadas en LLM como juez, adaptadas a artefactos de ingeniería de requisitos.

Para evaluar la viabilidad del enfoque propuesto, se realizan experimentos sobre el conjunto de datos Salony utilizando varios LLM de código abierto. Los resultados permiten analizar tanto la calidad de las descomposiciones generadas como el comportamiento de las estrategias de evaluación comparativa entre modelos.

El resto del artículo se organiza de la siguiente manera. La Sección 2 revisa el trabajo relacionado sobre automatización basada en LLM en ingeniería de requisitos y generación de datos sintéticos. La Sección 3 describe la arquitectura y los componentes principales de LocalLLM-DataForge. La Sección 4 presenta la configuración experimental y los resultados obtenidos. La Sección 5 discute los hallazgos y sus implicaciones, mientras que la Sección 6 expone las conclusiones y las líneas de trabajo futuro.

2 Trabajos Relacionados

La automatización de actividades dentro de la ingeniería de requisitos ha sido objeto de creciente interés en los últimos años, especialmente con el avance de técnicas NLP y, más recientemente, de los LLMs. Diversos estudios han explorado el potencial de estos modelos para asistir en tareas tradicionalmente engorrosas en lenguaje natural (e.g., elicitación, análisis, reformulación y validación de requisitos). Investigaciones recientes muestran que los LLMs pueden mejorar la calidad de los artefactos de requisitos y facilitar la automatización parcial de diferentes actividades del proceso de ingeniería de requisitos, incluyendo la generación de historias de usuario, la clasificación de requisitos y la evaluación de su calidad textual [16, 17]. En particular, algunos trabajos han demostrado la viabilidad de utilizar LLMs para generar artefactos derivados de requisitos o asistir en su análisis semántico dentro de flujos de trabajo de desarrollo de software [18, 19]. Estos avances sugieren que los modelos generativos pueden actuar como asistentes inteligentes en etapas tempranas del ciclo de vida del software, aunque su integración sistemática en procesos de ingeniería de requisitos aún se encuentra en una fase exploratoria.

Paralelamente, la generación de datos sintéticos mediante modelos generativos ha emergido como una estrategia prometedora para abordar la escasez de conjuntos de datos anotados en múltiples dominios. Los LLMs permiten producir grandes volúmenes de datos textuales controlados mediante prompts, lo que facilita la creación de datasets para entrenamiento, evaluación o ampliación de datos existentes. Estudios recientes han analizado el uso de LLMs para generar, curar y evaluar datos sintéticos, destacando su potencial para complementar o sustituir parcialmente conjuntos de datos reales cuando estos son limitados o costosos de obtener [20]. Aplicaciones de este enfoque se han observado en diversos contextos, incluyendo la generación automática de datasets en dominios especializados como análisis forense digital o sistemas legales basados en Inteligencia Artificial (AI, por sus siglas en inglés)[21, 22]. No obstante, la literatura también señala desafíos importantes relacionados con la calidad, coherencia y fiabilidad de los datos generados sintéticamente, lo que hace necesario desarrollar mecanismos sistemáticos de validación y control de calidad.

En este contexto, ha surgido recientemente el paradigma *LLM-as-a-Judge*, en el cual los modelos de lenguaje se utilizan no solo como generadores de contenido, sino también como evaluadores automáticos de las salidas producidas por otros modelos. Este enfoque ha sido adoptado en múltiples estudios de evaluación de modelos generativos, donde los LLMs actúan como evaluadores capaces de analizar criterios semánticos complejos que no pueden capturarse fácilmente mediante métricas automáticas tradicionales [23]. Investigaciones recientes han demostrado que este tipo de evaluación puede aproximar razonablemente los juicios humanos en tareas de generación de lenguaje natural, especialmente cuando se utilizan criterios explícitos o esquemas de comparación entre salidas [24]. Sin embargo, el uso de LLMs como evaluadores también plantea desafíos metodológicos, incluyendo posibles sesgos, dependencia de los prompts y riesgos de circularidad cuando los modelos se evalúan entre sí.

A pesar de estos avances, la literatura actual presenta todavía una brecha en la integración de estas tres líneas de investigación. En particular, existen pocos enfoques

que combinen generación de datos sintéticos, ejecución local de modelos y mecanismos estructurados de evaluación automática dentro de un mismo framework orientado a artefactos de ingeniería de requisitos. LocalLLM-DataForge aborda esta brecha mediante un framework diseñado para integrar estos elementos en un entorno capaz de generar y evaluar descomposiciones de historias de usuario de forma sistemática.

3 Framework LocalLLM-DataForge

LocalLLM-DataForge es un framework diseñado para la generación, transformación y evaluación automática de artefactos de ingeniería de requisitos mediante modelos de lenguaje ejecutados localmente. Su propósito principal es proporcionar una arquitectura modular, reproducible y extensible que permita integrar múltiples etapas de procesamiento dentro de un mismo flujo de trabajo.

A diferencia de enfoques tradicionales centrados en el entrenamiento de modelos, LocalLLM-DataForge se enfoca en la orquestación de modelos preentrenados para la producción controlada de salidas y su evaluación sistemática. De esta forma se facilita la experimentación reproducible en entornos sin dependencia de servicios en la nube.

3.1 Arquitectura General

El framework se estructura como un pipeline de procesamiento basado en DataFrames, donde cada componente representa una transformación explícita sobre los datos de entrada. Este enfoque permite modelar el flujo de trabajo como una secuencia de operaciones deterministas y trazables, en las que cada etapa produce nuevas representaciones sin alterar los datos originales. Como resultado, se favorece la auditabilidad del proceso, la inspección intermedia de resultados y la reproducibilidad experimental.

La elección de un modelo basado en DataFrames responde a la necesidad de manejar simultáneamente múltiples versiones de los artefactos generados. En particular, cada historia de usuario puede dar lugar a diversas descomposiciones producidas por distintos modelos o configuraciones, las cuales son almacenadas como columnas independientes dentro de la misma estructura. Esta organización permite realizar comparaciones sistemáticas sin requerir estructuras externas adicionales ni procesos de sincronización complejos.

El flujo general del sistema sigue cinco etapas principales: (1) carga de datos, (2) generación de artefactos, (3) comparación y evaluación, (4) obtención de métricas, y (5) almacenamiento de resultados. Estas etapas no se implementan como bloques monolíticos, sino como componentes desacoplados que operan sobre el mismo DataFrame, lo que permite reorganizar, extender o reemplazar partes del pipeline sin afectar el resto del sistema.

Un aspecto clave de la arquitectura es la separación explícita entre generación y evaluación. Mientras que los modelos generadores se encargan de producir descomposiciones a partir de historias de usuario, el componente evaluador opera de forma independiente para analizar la calidad de dichas salidas bajo criterios estructurados. Esta separación permite experimentar con configuraciones heterogéneas de modelos sin introducir sesgos en la evaluación, además de facilitar la incorporación de nuevos mecanismos de validación en futuras extensiones del framework.

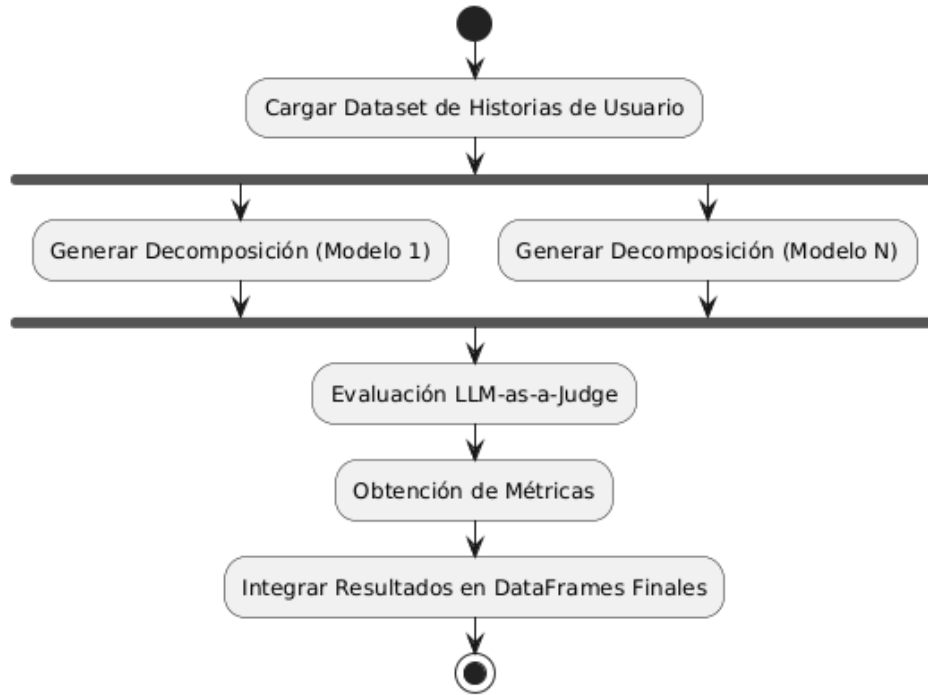


Fig. 1 Diagrama de actividad UML del framework LocalLLM-DataForge.

La Figura 1 representa el comportamiento general del sistema modelado mediante un diagrama de actividad UML, permitiendo representar explícitamente el flujo de procesamiento y la ejecución paralela de los modelos generadores. Este inicia con la carga del dataset de historias de usuario, seguido por una etapa de generación paralela en la que múltiples modelos producen descomposiciones independientes bajo configuraciones diferenciadas. Posteriormente, un componente evaluador basado en el paradigma *LLM-as-a-Judge* compara las salidas generadas y asigna las métricas calculadas. Finalmente, los resultados son integrados en el DataFrame de salida, permitiendo su análisis posterior de forma sistemática.

Este diseño arquitectónico prioriza la modularidad, la transparencia en el flujo de datos y la capacidad de experimentación controlada, aspectos fundamentales en estudios orientados a evaluar el comportamiento de modelos de lenguaje en tareas de ingeniería de requisitos.

3.2 Modelo de Pipeline

El núcleo operativo del framework es la clase `DataForgePipeline`, la cual implementa un modelo de ejecución basado en la composición secuencial de pasos modulares. En este enfoque, cada paso se define como una función de transformación que recibe

un DataFrame de entrada y produce un nuevo DataFrame enriquecido, preservando las columnas originales y agregando nuevas representaciones derivadas. Este tipo de organización es consistente con modelos de procesamiento basados en dataflow, donde el cómputo se expresa como una secuencia de transformaciones sobre los datos [25].

Este modelo puede interpretarse como una forma de *dataflow pipeline*, donde el estado del sistema se materializa explícitamente en cada etapa mediante estructuras tabulares. A diferencia de arquitecturas imperativas tradicionales, en las que las transformaciones pueden sobrescribir información previa, el diseño adoptado en LocalLLM-DataForge favorece la inmutabilidad lógica de los datos, permitiendo rastrear el origen y evolución de cada artefacto generado a lo largo del proceso.

Una característica distintiva del pipeline es su naturaleza declarativa. Los experimentos se definen especificando una secuencia ordenada de pasos y sus configuraciones, sin necesidad de gestionar explícitamente el flujo de control interno. Este enfoque contrasta con prácticas tradicionales basadas en scripts ad-hoc, las cuales dificultan la reproducibilidad y la reutilización de configuraciones experimentales. En este sentido, frameworks recientes para procesamiento de datos con LLMs han destacado la importancia de abstracciones modulares y pipelines configurables para mejorar la trazabilidad y reproducibilidad de los experimentos [26].

Desde una perspectiva de ingeniería, este modelo promueve un alto grado de desacoplamiento entre componentes. Cada paso encapsula su lógica de procesamiento y se comunica con el resto del sistema exclusivamente a través del DataFrame compartido. Este tipo de diseño modular ha sido ampliamente adoptado en sistemas modernos de procesamiento de datos y aprendizaje automático, donde la composición de transformaciones reutilizables permite construir flujos de trabajo complejos de manera flexible [26].

Adicionalmente, el pipeline soporta la ejecución incremental y la inspección intermedia de resultados. Dado que cada transformación produce una nueva versión del DataFrame, es posible analizar el estado del sistema en cualquier punto del flujo, lo cual resulta particularmente útil en tareas de depuración, validación de prompts y análisis cualitativo de salidas generadas por modelos de lenguaje.

En conjunto, el modelo de pipeline adoptado en LocalLLM-DataForge proporciona una base estructurada para la orquestación de modelos LLMs en tareas de generación y evaluación. De esta forma se alinea con tendencias recientes en el diseño de sistemas LLMs multi-etapa que integran componentes generativos y evaluadores dentro de un mismo flujo de procesamiento [27].

3.3 Componentes del Sistema

El framework incorpora un conjunto de componentes especializados que encapsulan funcionalidades específicas dentro del pipeline. Este diseño sigue principios de arquitecturas tipo *pipes-and-filters*, en las que el procesamiento se descompone en una secuencia de etapas independientes que transforman progresivamente los datos de entrada. En este tipo de arquitectura, cada componente, o filtro, realiza una operación específica y transmite su salida al siguiente componente a través de canales definidos [28]. Este enfoque ha sido ampliamente adoptado en sistemas de procesamiento de

datos y pipelines de aprendizaje automático, donde la separación de responsabilidades permite construir flujos complejos a partir de transformaciones simples y desacopladas.

Dentro de este esquema, el componente `LoadDataFrame` actúa como punto de entrada del pipeline, encargándose de la ingesta de datos estructurados y su normalización en una representación tabular unificada. Su función es establecer un estado inicial consistente que permita aplicar transformaciones posteriores sin ambigüedad estructural. Por su parte, el componente `OllamaLLMStep` encapsula la interacción con modelos de lenguaje ejecutados localmente, permitiendo parametrizar su comportamiento mediante configuraciones explícitas como temperatura, tamaño de lote y número máximo de tokens. Este componente materializa las salidas generadas por los modelos como nuevas columnas dentro del `DataFrame`, integrando de manera directa los artefactos generados en el flujo de procesamiento. La incorporación de modelos de lenguaje dentro de pipelines estructurados constituye una tendencia reciente en sistemas multi-etapa, donde múltiples modelos colaboran para resolver tareas complejas mediante generación y transformación progresiva de información [29].

El componente `ComparisonJudgeStep` implementa un mecanismo de evaluación basado en el paradigma *LLM-as-a-Judge*, en el cual un modelo de lenguaje es utilizado como evaluador de las salidas generadas por otros modelos. Este enfoque ha ganado relevancia como alternativa a métricas tradicionales en tareas abiertas, permitiendo aproximar juicios cualitativos mediante evaluaciones estructuradas y reproducibles [30]. En este contexto, el componente recibe múltiples candidatos generados y produce métricas de calidad que son integradas directamente en el `DataFrame`, lo que permite su análisis posterior sin requerir procesamiento adicional.

Estos componentes conforman una arquitectura en la que cada etapa del procesamiento se encuentra claramente definida y desacoplada, desde la generación hasta la evaluación. Esta separación de responsabilidades permite no solo extender el sistema mediante la incorporación de nuevos módulos, sino también experimentar con distintas configuraciones de modelos y estrategias de evaluación sin introducir dependencias rígidas, alineándose con principios fundamentales de diseño en arquitecturas orientadas a procesamiento de datos.

3.4 Ejecución Local y Gestión de Recursos

Una característica central de `LocalLLM-DataForge` es su enfoque en la ejecución local de modelos de lenguaje, lo que permite mantener un control completo sobre los datos procesados y los recursos computacionales utilizados. A diferencia de los enfoques basados en servicios en la nube, donde las entradas y salidas pueden ser almacenadas o procesadas por terceros, la ejecución local garantiza que toda la información permanezca dentro del entorno del usuario, lo cual resulta particularmente relevante en contextos donde se manejan datos sensibles o sujetos a regulaciones de privacidad [31]. Este aspecto ha sido identificado en la literatura como una de las principales ventajas de los modelos de código abierto y despliegue local, ya que elimina riesgos asociados a la exposición de datos y mejora la transparencia del procesamiento [32].

Desde una perspectiva experimental, la ejecución local también contribuye significativamente a la reproducibilidad de los resultados. En entornos basados en APIs

externas, factores como cambios en versiones de modelos, ajustes internos no documentados o variaciones en la infraestructura pueden afectar los resultados obtenidos. En contraste, el uso de modelos locales permite fijar versiones específicas, configuraciones y condiciones de ejecución, asegurando que los experimentos puedan ser replicados bajo las mismas condiciones [32]. Esta propiedad es especialmente importante en estudios orientados a evaluar el comportamiento de modelos de lenguaje, donde pequeñas variaciones en el entorno pueden impactar de manera significativa las salidas generadas.

No obstante, la ejecución local introduce también desafíos asociados a la gestión eficiente de recursos computacionales. Los modelos de lenguaje requieren cantidades significativas de memoria y capacidad de procesamiento, lo que obliga a diseñar mecanismos que optimicen su uso en hardware limitado. En este contexto, el framework implementa un esquema de procesamiento por lotes configurable que permite controlar el número de instancias procesadas simultáneamente, evitando desbordamientos del contexto y reduciendo la presión sobre la memoria disponible. Este enfoque permite adaptar dinámicamente la ejecución del pipeline a las capacidades del sistema, manteniendo un equilibrio entre rendimiento y estabilidad operativa.

Adicionalmente, se incorpora un sistema de caché opcional que permite reutilizar resultados previamente generados cuando las entradas y configuraciones no han cambiado. Esta estrategia resulta particularmente útil en escenarios de experimentación iterativa, donde múltiples ejecuciones con parámetros similares son comunes. La reutilización de resultados no solo reduce el tiempo de ejecución, sino que también contribuye a la consistencia de los experimentos al evitar variaciones innecesarias en las salidas generadas.

En conjunto, el enfoque de ejecución local adoptado en LocalLLM-DataForge no solo responde a consideraciones de privacidad y control de datos, sino que también constituye una decisión metodológica orientada a garantizar reproducibilidad, trazabilidad y estabilidad en el proceso experimental, aspectos fundamentales en la evaluación rigurosa de sistemas basados en modelos de lenguaje.

3.5 Diseño de Prompts

El diseño de prompts constituye un componente metodológico central dentro del framework, ya que define la forma en que los modelos de lenguaje interpretan las tareas y generan sus respuestas. En el contexto de los LLMs, el prompt puede entenderse como un mecanismo de programación implícita, mediante el cual es posible guiar el comportamiento del modelo sin necesidad de modificar sus parámetros internos [33]. Diversos estudios han demostrado que la calidad, estructura y claridad del prompt influyen directamente en la precisión, consistencia y utilidad de las salidas generadas, posicionando al prompt engineering como una disciplina clave en el aprovechamiento efectivo de estos modelos [34].

En LocalLLM-DataForge, los prompts se diseñan siguiendo un enfoque estructurado basado en esquemas de instrucción–entrada–respuesta, en el que se especifican explícitamente tanto las restricciones de contenido como el formato esperado de salida. Este tipo de estructuración ha sido identificado como una estrategia efectiva para

mejorar la consistencia de los resultados y facilitar su procesamiento automático, especialmente en tareas que requieren generar datos estructurados [35]. En particular, la imposición de formatos rígidos permite reducir la ambigüedad en las salidas y habilita su integración directa en el pipeline sin necesidad de etapas adicionales de limpieza o normalización.

Dentro del framework se emplean dos prompts principales: (i) un *prompt generador de tareas*, encargado de descomponer historias de usuario en conjuntos estructurados de tareas, y (ii) un *prompt de evaluación bajo el enfoque LLM-as-a-Judge*, orientado a valorar la calidad de dichas descomposiciones conforme a criterios definidos. Ambos prompts son diseñados bajo principios homogéneos de estructuración y control, pero responden a objetivos funcionales distintos dentro del pipeline. Con el fin de garantizar la reproducibilidad del experimento y la transparencia metodológica, las versiones completas de estos prompts se presentan en el Apéndice A y el Apéndice B, respectivamente.

En las tareas de generación, el prompt generador está diseñado para producir descomposiciones de historias de usuario bajo restricciones específicas, tales como la no superposición de tareas, la claridad en la redacción y la inclusión de campos estructurados como resumen y descripción. Estas restricciones actúan como mecanismos de control que delimitan el espacio de salida del modelo, favoreciendo la producción de artefactos consistentes y comparables. Por otro lado, en las tareas de evaluación, el prompt basado en LLM-as-a-Judge define explícitamente criterios de calidad y reglas de decisión, instruyendo al modelo para emitir juicios estructurados en formatos como JSON. Este enfoque permite transformar evaluaciones cualitativas en representaciones cuantificables que pueden ser integradas directamente en el flujo de procesamiento.

Adicionalmente, el uso de prompts estandarizados entre diferentes modelos permite realizar comparaciones más justas y controladas, evitando sesgos derivados de ajustes específicos para cada modelo. La literatura reciente ha señalado que la consistencia en el diseño de prompts es un factor crítico para garantizar la validez de evaluaciones comparativas entre LLMs, ya que pequeñas variaciones en la formulación pueden impactar significativamente los resultados obtenidos [35]. En este sentido, el framework adopta un enfoque uniforme en la definición de prompts, asegurando que las diferencias observadas en las salidas se deban principalmente a las características de los modelos y no a variaciones en las instrucciones proporcionadas.

Es por esto que el diseño de prompts en LocalLLM-DataForge no se limita a una tarea de configuración, sino que constituye un elemento fundamental del proceso experimental, al actuar como interfaz de control entre el investigador y el modelo. Este enfoque permite ejercer un control fino sobre la generación y evaluación de artefactos, alineándose con tendencias recientes que conciben el prompt engineering como una capa de abstracción clave en sistemas basados en modelos de lenguaje.

4 Resultados

Esta sección presenta los resultados del estudio experimental realizado para evaluar el desempeño del *framework* propuesto en la generación automática de descomposiciones de historias de usuario. En primer lugar, se describe la configuración experimental

empleada, incluyendo el entorno de ejecución, los modelos utilizados y el esquema de evaluación. Posteriormente, se presentan los resultados agregados obtenidos en las distintas pruebas, seguidos de un análisis detallado por criterio que permite identificar patrones de comportamiento y diferencias entre modelos. Finalmente, se discuten los hallazgos más relevantes derivados del estudio.

4.1 Configuración experimental

El estudio experimental se realizó utilizando la herramienta *LocalLLM-DataForge*, con el objetivo de evaluar la generación automática de descomposiciones de historias de usuario mediante modelos de lenguaje ejecutados localmente.

El entorno consistió en un servidor Dell equipado con un procesador AMD EPYC 9124 a 3.0 GHz (16 núcleos, 32 hilos, 64 MB de caché), con 64 GB de memoria RAM DDR5-4800 (4 módulos de 16 GB) y una GPU NVIDIA Ampere A2 con 16 GB de memoria. El sistema se ejecutó utilizando Python v3.13.7 y Ollama v0.21.2 como plataforma de orquestación para la ejecución local de los modelos. La GPU fue utilizada para acelerar la inferencia de los modelos, permitiendo reducir los tiempos de generación y evaluación durante las pruebas experimentales.

El conjunto de datos base fue Salony¹, del cual se seleccionaron 1,139 de 1,998 historias de usuario que cumplieran con la estructura canónica “*As a(n) [role], I want [feature] so that [benefit]*”. Cada historia fue procesada por dos modelos generadores (G1 y G2), configurados con diferentes parámetros de generación para fomentar comportamientos contrastantes. Las descomposiciones generadas fueron evaluadas mediante un modelo bajo el paradigma *LLM-as-a-Judge*, asignando puntuaciones en cinco criterios: coherencia, completitud, viabilidad, formato y granularidad [36][37][38].

La selección y definición de estos criterios se fundamenta en los principios de calidad de requisitos establecidos por la norma ISO/IEC/IEEE 29148:2018 [43], la cual enfatiza características como la claridad, consistencia, completitud y verificabilidad de los artefactos de requisitos. En este sentido, la coherencia evaluó la correspondencia semántica entre las tareas generadas y la historia de usuario original, alineándose con la propiedad de consistencia. La completitud midió el grado de cobertura de los requerimientos implícitos y explícitos, en concordancia con el atributo de completitud definido por la norma. La viabilidad analizó si las tareas propuestas eran técnicamente realizables, lo cual se relaciona con la verificabilidad y factibilidad de los requisitos. El criterio de formato verificó la estructura y claridad de la salida, en línea con la necesidad de que los requisitos sean comprensibles y no ambiguos. Finalmente, la granularidad evaluó la adecuación del nivel de descomposición, apoyando la correcta especificación y atomicidad recomendada en la ingeniería de requisitos.

Cada criterio fue evaluado en una escala de 0 a 10, con una puntuación máxima de 50 puntos [41][42]. Se estableció un umbral de aprobación de 35 puntos (70%), permitiendo clasificar automáticamente cada instancia como válida o no válida [45].

¹Salony: User Story Dataset disponible en Hugging Face, https://huggingface.co/datasets/salony/User_story.

Además de la puntuación total, se registraron métricas agregadas como media, desviación estándar, tasa de aprobación y tasa de victoria en comparaciones directas entre modelos. Este esquema permitió analizar el desempeño relativo de los generadores bajo distintas configuraciones.

4.2 Resultados globales

Con el objetivo de evaluar el comportamiento del *framework* propuesto bajo distintas configuraciones de modelos de lenguaje, se diseñó un estudio experimental en torno a la comparación entre dos modelos generadores de descomposiciones de historias de usuario (G1 y G2), junto con un modelo evaluador que actuó bajo el paradigma *LLM-as-a-Judge*.

Las configuraciones evaluadas consideran modelos de diferentes familias, tamaños y arquitecturas, con el fin de analizar su impacto en la generación y evaluación de descomposiciones de historias de usuario. La Tabla 1 muestra las combinaciones utilizadas en cada prueba, incluyendo los modelos generadores y el modelo evaluador correspondiente, lo que permite analizar tanto el efecto de los generadores como del modelo juez.

Table 1 Configuraciones de modelos evaluadas en las pruebas experimentales

Test	Rol	Modelo	Arquitectura	Tamaño
Test 1	G1	Llama 3.1	Transformer (decoder)	8B
	G2	Mistral	Transformer (decoder)	7B
	Judge	Qwen 3	Transformer (decoder)	14B
Test 2	G1	Qwen 3	Transformer (decoder)	8B
	G2	Gemma 4 (E4B)	Transformer (decoder)	4B
	Judge	Qwen 2.5	Transformer (decoder)	14B
Test 3	G1	Gemma 3	Transformer (decoder)	12B
	G2	Mistral Nemo	Transformer (decoder)	12B
	Judge	Qwen 3	Transformer (decoder)	14B
Test 4	G1	Qwen 3	Transformer (decoder)	14B
	G2	Phi-4	Transformer (decoder)	14B
	Judge	GPT-OSS	Transformer (decoder)	20B

En cada configuración, ambos generadores procesaron el mismo conjunto de historias de usuario, produciendo descomposiciones alternativas para cada instancia. Este filtrado permitió garantizar la consistencia en la entrada de los modelos y la validez de la evaluación. Posteriormente, el modelo evaluador analizó ambas salidas y asignó puntuaciones en los cinco criterios de calidad para determinar un ganador entre las dos alternativas. De esta forma se consigue no solo un análisis cuantitativo basado en puntuaciones, sino también una comparación directa entre modelos.

Las cuatro pruebas se diseñaron variando tanto los modelos generadores como el modelo evaluador, con el fin de analizar el impacto de estas configuraciones en la

calidad de las descomposiciones generadas. En particular, se consideraron modelos con diferentes tamaños, arquitecturas y configuraciones de generación, lo que permitió observar comportamientos contrastantes en términos de consistencia, diversidad y alineación con los criterios de evaluación.

Para cada prueba se calcularon métricas agregadas, incluyendo puntuación promedio, desviación estándar, tasa de aprobación (definida como el porcentaje de instancias que superan el umbral de calidad establecido) y tasa de victoria en comparaciones directas. Asimismo, se realizó un análisis desagregado por criterio, con el objetivo de identificar fortalezas y debilidades específicas en las descomposiciones generadas por cada modelo.

Con el objetivo de identificar patrones consistentes en el comportamiento de los modelos evaluados, se realizó un análisis comparativo considerando de manera conjunta los resultados de las pruebas experimentales. Este enfoque permite observar tendencias globales en términos de desempeño, estabilidad y comportamiento relativo entre modelos.

La Tabla 2 presenta las métricas agregadas para cada prueba, incluyendo puntuación promedio (*Mean*), desviación estándar (*Std*), tasa de aprobación (*Pass*) y tasa de victoria en comparaciones directas (*Win*).

Table 2 Resultados agregados por prueba

Test	G1 Mean	G1 Std	G1 Pass	G2 Mean	G2 Std	G2 Pass	G1 Win	G2 Win
Test 2	35.70	9.88	55.22	35.95	10.26	55.22	72.78	27.22
Test 3	35.70	9.88	55.22	35.95	10.26	55.22	72.78	27.22

Los resultados muestran una alta consistencia entre las pruebas consideradas. En particular, las puntuaciones promedio de ambos modelos son prácticamente equivalentes en todos los casos, con diferencias mínimas. De manera similar, las tasas de aprobación coinciden entre modelos dentro de cada prueba, lo que indica un nivel de desempeño global comparable. En contraste, la tasa de victoria presenta un patrón diferenciado, donde G1 es seleccionado como mejor alternativa en aproximadamente el 73% de los casos en ambas pruebas.

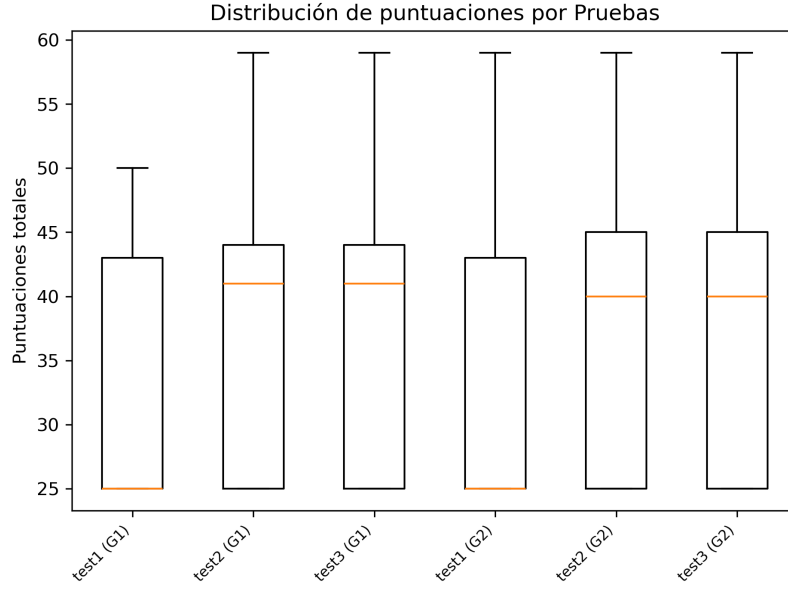


Fig. 2 Distribución de puntuaciones totales por prueba y modelo.

La Figura 2 muestra la distribución de puntuaciones totales para cada modelo en las distintas pruebas. Estas presentan un alto grado de solapamiento entre modelos en todas las pruebas, lo que refleja la similitud en sus desempeños globales. Asimismo, se observa una dispersión considerable en las puntuaciones, evidenciada por la amplitud de los rangos intercuartílicos y la extensión de los valores extremos, lo que indica variabilidad en la calidad de las descomposiciones generadas.

4.3 Análisis por criterio

En la Tabla 3 se presentan las puntuaciones promedio por criterio para cada prueba.

Table 3 Puntuación promedio por criterio en las pruebas

Criterio	Test 2 (G1)	Test 2 (G2)	Test 3 (G1)	Test 3 (G2)
Coherencia	4.92	4.79	4.92	4.79
Compleitud	4.57	4.88	4.57	4.88
Viabilidad	4.97	4.94	4.97	4.94
Formato	5.26	5.26	5.26	5.26
Granularidad	4.47	4.57	4.47	4.57

El análisis por criterio muestra valores muy cercanos entre modelos en todas las dimensiones evaluadas. En coherencia y viabilidad, G1 presenta puntuaciones ligeramente superiores, mientras que en completitud y granularidad G2 obtiene valores

mayores. En el criterio de formato, ambos modelos alcanzan prácticamente el mismo nivel. Estas diferencias se mantienen constantes entre las pruebas analizadas.

La Figura 3 muestra la comparación de los criterios, lo que confirma que las diferencias entre modelos son reducidas y consistentes entre pruebas. Esto no permite observar una dominación clara de alguno de los modelos sobre los otros en los criterios evaluados. En conjunto, los resultados muestran que ambos modelos presentan comportamientos similares en términos de desempeño global, con variaciones pequeñas pero sistemáticas en criterios específicos y en la selección del mejor resultado.

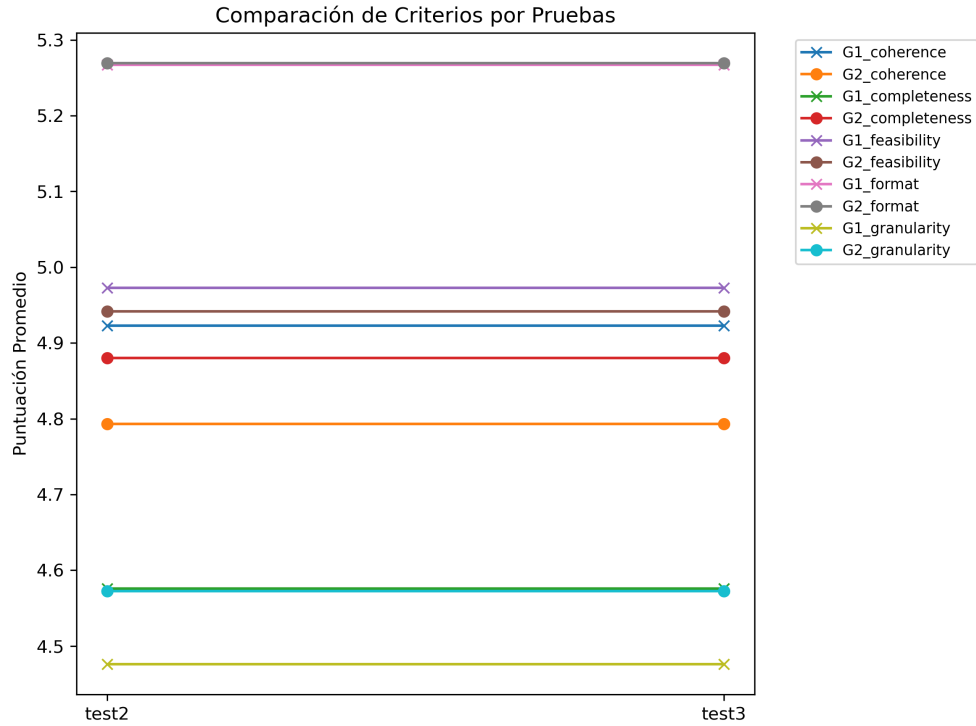


Fig. 3 Comparación de criterios por prueba y modelo.

5 Discusión

6 Conclusión

Agradecimientos.

Appendix A Prompt de Generación de Tareas

Este apéndice presenta el prompt utilizado para la generación automática de tareas a partir de historias de usuario. Dicho prompt sigue un esquema de instrucción-entrada-respuesta, en el cual se especifican explícitamente las restricciones de descomposición, así como el formato estructurado esperado en la salida.

Listing 1 Prompt generador de tareas

```
Below is an instruction that describes a task, paired with an
input that provides a user story.
```

```
Write a response that appropriately completes the request.
```

```
Instruction:
```

```
Break this user story into smaller development tasks to help the
developers implement it efficiently. You can divide this user
story into as many tasks as needed, depending on its
complexity. Each task must be unique, actionable, and non-
overlapping.
```

```
Use the following format for the response:
```

```
1. summary: <task summary 1>
description: <task description 1>
2. summary: <task summary 2>
description: <task description 2>
```

```
N. summary: <task summary N>
description: <task description N>
```

```
Input:
```

```
{user_story}
```

```
Response:
```

Appendix B Prompt de Evaluación LLM-as-a-Judge

En este apéndice se presenta el prompt utilizado para la evaluación automática de descomposiciones de historias de usuario bajo el enfoque *LLM-as-a-Judge*. Este prompt instruye al modelo para comparar dos alternativas de descomposición y emitir un juicio basado en criterios de calidad alineados con el estándar ISO/IEC/IEEE 29148:2018.

Listing 2 Prompt LLM-as-a-Judge

```
You are an expert software development manager evaluating two
different breakdowns of a user story into development tasks.
```


Your job is to compare both breakdowns and determine which one is better overall.

USER STORY:
{user_story}

BREAKDOWN A:
{tasks_a}

BREAKDOWN B:
{tasks_b}

Evaluate both breakdowns based on these criteria (0-10 points each), aligned with ISO/IEC/IEEE 29148:2018 quality standards:

1. COHERENCE: Semantic correspondence between the generated tasks and the original user story. Does it maintain consistency with the user story intent? (0-10)
2. COMPLETENESS: Degree of coverage of implicit and explicit requirements. Does it cover all necessary aspects of the user story? (0-10)
3. FEASIBILITY: Technical feasibility of the proposed tasks. Are the tasks realistically implementable? (0-10)
4. FORMAT: Structure and clarity of the output. Is the response well-formatted, unambiguous, and easy to understand? (0-10)
5. GRANULARITY: Appropriateness of the decomposition level. Are tasks properly sized (not too big, not too small) and atomic? (0-10)

Respond in this exact JSON format:

```
{
  "breakdown_a": {
    "coherence": [score_0_to_10],
    "completeness": [score_0_to_10],
    "feasibility": [score_0_to_10],
    "format": [score_0_to_10],
    "granularity": [score_0_to_10],
    "total_score": [sum_of_all_scores],
    "strengths": "[brief_description_of_strengths]",
    "weaknesses": "[brief_description_of_weaknesses]"
  },
  "breakdown_b": {
    "coherence": [score_0_to_10],
    "completeness": [score_0_to_10],
    "feasibility": [score_0_to_10],
    "format": [score_0_to_10],
    "granularity": [score_0_to_10],
    "total_score": [sum_of_all_scores],
    "strengths": "[brief_description_of_strengths]",
    "weaknesses": "[brief_description_of_weaknesses]"
  }
}
```

```

},
"winner": "[A_or_B]",
"reason": "[brief_explanation_of_why_the_winner_is_better]"
}

```

References

- [1] Cohn, M.: User Stories Applied: For Agile Software Development. Addison Wesley Longman Publishing Co., Inc., USA (2004)
- [2] Beck, K.: Extreme Programming Explained: Embrace Change. Addison-Wesley Longman Publishing Co., Inc., USA (1999)
- [3] Rubin, K.S.: Essential Scrum: A Practical Guide to the Most Popular Agile Process, 1st edn. Addison-Wesley Professional, Boston, MA (2012)
- [4] Schwaber, K., Sutherland, J.: The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. Scrum.org & Scrum Inc. (2020). <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>
- [5] Lucassen, G., Dalpiaz, F., Van Der Werf, J.M., Brinkkemper, S.: Improving agile requirements: the quality user story framework and tool. *Requir. Eng.* **21**(3), 383–403 (2016) <https://doi.org/10.1007/s00766-016-0250-x>
- [6] Gorschek, T., Garre, P., Larsson, S., Wohlin, C.: A model for technology transfer in practice. *IEEE Softw.* **23**(6), 88–95 (2006) <https://doi.org/10.1109/MS.2006.147>
- [7] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D.W., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Babuschkin, I., Balaji, S., Jain, S., Carr, A., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M.M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W.: Evaluating large language models trained on code. *ArXiv abs/2107.03374* (2021)
- [8] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C.: A prompt pattern catalog to enhance prompt engineering with chatgpt. In: *Proceedings of the 30th Conference on Pattern Languages of Programs. PLoP '23*. The Hillside Group, USA (2023)
- [9] Zhao, L., Alhoshan, W., Ferrari, A., Letsholo, K.J., Ajagbe, M.A., Chioasca, E.-V., Batista-Navarro, R.T.: Natural language processing for requirements engineering: A systematic mapping study. *ACM Comput. Surv.* **54**(3) (2021) <https://doi.org/10.1145/3456789>

[//doi.org/10.1145/3444689](https://doi.org/10.1145/3444689)

- [10] Zadenoori, M.A., Dabrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large language models (llms) for requirements engineering (re): A systematic literature review. *ArXiv abs/2509.11446* (2025)
- [11] Alhaizaey, A., Al-Mashari, M.: An aggregated dataset of agile user stories and use case taxonomy for ai-driven research. *International Journal of Advanced Computer Science and Applications* **16**(9) (2025) <https://doi.org/10.14569/IJACSA.2025.0160925>
- [12] Nadăș, M., Diosan, L., Tomescu, A.: Synthetic data generation using large language models: Advances in text and code. *IEEE Access* **PP**, 1–1 (2025) <https://doi.org/10.1109/ACCESS.2025.3589503>
- [13] Ho, N., Schmid, L., Yun, S.-Y.: Large language models are reasoning teachers. In: *Annual Meeting of the Association for Computational Linguistics* (2022). <https://api.semanticscholar.org/CorpusID:254877399>
- [14] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E.H., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. *ArXiv abs/2203.11171* (2022)
- [15] Li, Y., Lin, Z., Zhang, S.D., Fu, Q., Chen, B., Lou, J.-G., Chen, W.: Making language models better reasoners with step-aware verifier. In: *Annual Meeting of the Association for Computational Linguistics* (2022). <https://api.semanticscholar.org/CorpusID:259370847>
- [16] Niv, S.: Leveraging the performance of large language models in systems engineering applications. *Journal of Information Systems Engineering and Management* **10**, 677–687 (2025) <https://doi.org/10.52783/jisem.v10i27s.4524>
- [17] Vogelsang, A., Fischbach, J.: In: Ferrari, A., Ginde, G. (eds.) *Using Large Language Models for Natural Language Processing Tasks in Requirements Engineering: A Systematic Guideline*, pp. 435–456. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-73143-3_16 . https://doi.org/10.1007/978-3-031-73143-3_16
- [18] Norheim, J.J., Rebentisch, E., Xiao, D., Draeger, L., Kerbrat, A., Weck, O.L.: Challenges in applying large language models to requirements engineering tasks. *Design Science* **10**, 16 (2024) <https://doi.org/10.1017/dsj.2024.8>
- [19] Ferrari, A., Spoletini, P.: Formal requirements engineering and large language models: A two-way roadmap. *Inf. Softw. Technol.* **181**(C) (2025) <https://doi.org/10.1016/j.infsof.2025.107697>

- [20] Li, Z., Zhu, H., Lu, Z., Yin, M.: Synthetic Data Generation with Large Language Models for Text Classification: Potential and Limitations. <https://doi.org/10.18653/v1/2023.emnlp-main.647> . <https://aclanthology.org/2023.emnlp-main.647/>
- [21] Chakir, Y., Lahsen-Cherif, I.: Forensicsdata: A digital forensics dataset for large language models. In: 2025 21th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), pp. 1–6 (2025). <https://doi.org/10.1109/WiMob66857.2025.11257556>
- [22] Ma, H., Lu, Y., Xiao, Z., Feng, J., Zhang, H., Yu, J.: Sdd-lawllm: Advancing intelligent legal systems through synthetic data-driven fine-tuning of large language models. *Electronics* **14**(4) (2025) <https://doi.org/10.3390/electronics14040742>
- [23] Do, H.J., Ashktorab, Z., Gajcin, J., Miehl, E., Cooper, M.S., Pan, Q., Daly, E.M., Geyer, W.: Generate, evaluate, iterate: Synthetic data for human-in-the-loop refinement of llm judges. *ArXiv abs/2511.04478* (2025)
- [24] Chirkova, N., Ajayi, T.O., Aycok, S., Mujahid, Z.M., Perlić, V., Borisova, E., Vartampetian, M.: LLM-as-a-qualitative-judge: automating error analysis in natural language generation. In: Chen, P., Zouhar, V., Hu, H., Khanuja, S., Zhu, W., Haddow, B., Birch, A., Aji, A.F., Sennrich, R., Hooker, S. (eds.) *Proceedings of the First Workshop on Multilingual Multicultural Evaluation*, pp. 99–132. Association for Computational Linguistics, Rabat, Morocco (2026). <https://doi.org/10.18653/v1/2026.mme-main.7> . <https://aclanthology.org/2026.mme-main.7/>
- [25] Lienhardt, M., ter Beek, M.H., Damiani, F.: Product lines of dataflows. *Journal of Systems and Software* **210**, 111928 (2024) <https://doi.org/10.1016/j.jss.2023.111928>
- [26] Liang, H., Ma, X., Liu, Z., Wong, Z.H., Zhao, Z., Meng, Z., He, R., Shen, C., Cai, Q., Han, Z., Qiang, M., Feng, Y., Bai, T., Pan, Z., Guo, Z., Jiang, Y., Deng, J., You, Q., Lai, P., Guo, T., Tsai, C.H., Feng, H., Hu, R., Yu, W.-t., Niu, J., Zeng, B., An, R., Ma, L., Huang, J., Zheng, Y., He, C., Tang, L., Cui, B., Weinan, E., Zhang, W.: Dataflow: An llm-driven framework for unified data preparation and workflow automation in the era of data-centric ai. *ArXiv abs/2512.16676* (2025)
- [27] Zhu, C., Hong, S., Wu, J., Chawla, K., Tang, C., Yin, Y., Wolfe, N., Babinsky, E., Liu, D.: Raffles: Reasoning-based attribution of faults for llm systems. In: *Conference of the European Chapter of the Association for Computational Linguistics* (2025). <https://api.semanticscholar.org/CorpusID:281204051>
- [28] Warnett, S.J., Ntontos, E., Zdun, U.: Mlops pipeline generation for reinforcement learning: A low-code approach using large language models. *Journal of Systems and Software* **235**, 112760 (2026) <https://doi.org/10.1016/j.jss.2025.112760>

- [29] Schnabel, J.A., Trippas, J.R., Scholer, F., Hettiachchi, D.: Multi-stage large language model pipelines can outperform gpt-4o in relevance assessment. Companion Proceedings of the ACM on Web Conference 2025 (2025)
- [30] Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, Y., Guo, J.: A survey on llm-as-a-judge. ArXiv **abs/2411.15594** (2024)
- [31] Kivimäki, T.: Usability evaluation of the local large language models. Master’s thesis, University of Turku, Turku, Finland (June 2025). <https://www.utupub.fi/handle/11111/20392>
- [32] Carammia, M., Iacus, S.M., Porro, G.: Rethinking scale: The efficacy of fine-tuned open-source llms in large-scale reproducible social science research. ArXiv **abs/2411.00890** (2024)
- [33] Vatsal, S., Dubey, H.: A survey of prompt engineering methods in large language models for different nlp tasks. ArXiv **abs/2407.12994** (2024)
- [34] Chen, B., Zhang, Z., Langrené, N., Zhu, S.: Unleashing the potential of prompt engineering for large language models. Patterns **6**(6), 101260 (2025) <https://doi.org/10.1016/j.patter.2025.101260>
- [35] White, J., Elnashar, A., Schmidt, D.: Prompt engineering for structured data a comparative evaluation of styles and llm performance. Preprints (2025) <https://doi.org/10.20944/preprints202506.1937.v1>
- [36] Baysan, M.S., Uysal, S., İşlek, , Çığ Karaman, , Güngör, T.: Llm-as-a-judge: automated evaluation of search query parsing using large language models. Frontiers in Big Data **Volume 8 - 2025** (2025) <https://doi.org/10.3389/fdata.2025.1611389>
- [37] PPCexpo Editorial Team: 10 Point Likert Scale: Everything You Need to Know. Accessed: 2026-04-26 (2024). <https://ppcexpo.com/blog/10-point-likert-scale>
- [38] Preston, C.C., Colman, A.M.: Optimal number of response categories in rating scales: reliability, validity, discriminating power, and respondent preferences. Acta Psychologica **104**(1), 1–15 (2000) [https://doi.org/10.1016/S0001-6918\(99\)00050-5](https://doi.org/10.1016/S0001-6918(99)00050-5)
- [39] Jebb, A.T., Ng, V., Tay, L.: A review of key likert scale development advances: 1995–2019. Frontiers in Psychology **12**, 637547 (2021) <https://doi.org/10.3389/fpsyg.2021.637547>
- [40] Tanujaya, B., Prahmana, R., Mumu, J.: Likert scale in social sciences research: Problems and difficulties. FWU Journal of Social Sciences **16**, 89–101 (2023) <https://doi.org/10.51709/19951272/Winter2022/7>

- [41] Wu, H., Leung, S.-O.: Can likert scales be treated as interval scales? *Journal of Social Service Research* **43**(4), 527–532 (2017) <https://doi.org/10.1080/01488376.2017.1329775>
- [42] Dawes, J.: Do data characteristics change according to the number of scale points used? *Int. J. Mark. Res.* **50**, 61–77 (2007)
- [43] Systems and Software Engineering — Life Cycle Processes — Requirements Engineering. International Organization for Standardization. <https://www.iso.org/standard/72089.html>
- [44] Guerdan, L., Barocas, S., Holstein, K., Wallach, H.M., Wu, Z.S., Choudhova, A.: Validating llm-as-a-judge systems under rating indeterminacy. (2025). <https://api.semanticscholar.org/CorpusID:276903119>
- [45] Consumer Council for Water: Research into threshold of acceptability. Technical report, Consumer Council for Water (CCW) (August 2013). Accessed: 2026-04-26. <https://www.ccw.org.uk/publication/research-into-threshold-of-acceptability/>
- [46] Wang, Y., Song, Y., Zhu, T., Zhang, X., Yu, Z., Chen, H., Song, C., Wang, Q., Wang, C., Wu, Z., Dai, X., Zhang, Y., Ye, W., Zhang, S.: Trustjudge: Inconsistencies of llm-as-a-judge and how to alleviate them. *ArXiv* **abs/2509.21117** (2025)